

ජර්ශනාවලිය (Questionnaire Module)

Author: Yasithzz | 2026

04 -- ජර්ශනාවලිය (Questionnaire / Lifestyle Input)

LifeMate වියාපෘතිය -- සිංහල ලේඛනය

1. Questionnaire Module යනු කුමක්ද?

LifeMate වියාපෘතියේ "Questionnaire" (ජර්ශනාවලිය) module -- LifestylePage.jsx (Lifestyle Data page) ලෙස frontend ලෙස කිරීමෙන්ම වේ. මෙය user ගේ දෛනික ජීවන රටාව (daily lifestyle data) ලබාගෙන ML model ශීර්ෂයට යවා, stress level prediction ලබා ගනී.

WellnessPage.jsx -- ජල (water intake) සහ වියායාම (workout) tracker, stress level අනුව personalized suggestions.

2. LifestylePage.jsx -- රේඛා රේඛා පැහැදිලි කිරීම

2.1 -- Imports සහ Constants

```
import { useEffect, useState } from 'react'
import { useNavigate } from 'react-router-dom'
import { Brain, Moon, Battery, ... } from 'lucide-react'
import { submitLifestyle, getLifestyleHistory, ... } from '../lib/api'
```

සිංහල:

- * useState -- React component state (form values, results, history).
- * useNavigate -- page navigation ශීර්ෂය.
- * lucide-react -- icons (Brain, Moon, Battery icons).
- * lib/api -- backend API call functions.

```
const MOODS = [
  { emoji: "😊", label: 'Great', value: 5 },
  { emoji: "🙂", label: 'Good', value: 4 },
  { emoji: "😐", label: 'Okay', value: 3 },
  { emoji: "😓", label: 'Low', value: 2 },
  { emoji: "😫", label: 'Stressed', value: 1 },
]
const WORKLOAD = ['Very Light','Light','Moderate','Heavy','Very Heavy']
const ENERGY = ['Exhausted','Tired','Okay','Energized','Very Energized']
const SOCIAL = ['None','Minimal','Moderate','Social','Very Social']
```

සිංහල: UI ලෙස mood, workload, energy, social interaction selection options. Emoji buttons ලෙස user friendly.

```
const STRESS_COLORS = {
  'Very Low': 'text-sky-400 bg-sky-500/15 border-sky-500/25',
  Low: 'text-emerald-400 bg-emerald-500/15 border-emerald-500/25',
  Normal: 'text-amber-400 bg-amber-500/15 border-amber-500/25',
  High: 'text-orange-400 bg-orange-500/15 border-orange-500/25',
  'Very High': 'text-red-400 bg-red-500/15 border-red-500/25',
}
```

සිංහල: Stress level අනුව result card color. High -- red. Very Low -- sky blue. User ඔහුගේ stress ශීර්ෂේඨ ලෙස visual ලෙස හඳුනාගත හැකි.

2.2 -- Component State

```
const [form, setForm] = useState({
  mood: 3, workload: 3, sleepHours: 7, energyLevel: 3,
  socialInteraction: 3, exerciseDone: 0, screenTimeHours: 4, waterCups: 6,
})
const [todayEntry, setTodayEntry] = useState(null)
const [history, setHistory] = useState([])
const [holidays, setHolidays] = useState([])
const [submitting, setSubmitting] = useState(false)
const [result, setResult] = useState(null)
```

සිංහල:

- * form -- 8 lifestyle inputs ගේ current values. Default: mood=3, sleep=7hrs, etc.
- * todayEntry -- අද දිනය submit කළ entry (ඇත් නම් "Submitted today" badge).
- * history -- ගිය දින lifestyle history.
- * holidays -- user holidays/leave dates.
- * submitting -- Submit button disabled (loading state).
- * result -- ML model prediction result.

2.3 -- useEffect (Page Load)

```
useEffect(() => {
  Promise.allSettled([getLifestyleToday(), getLifestyleHistory(), getHolidays()])
    .then(([t, h, hl]) => {
      if (t.status==='fulfilled' && t.value) { setTodayEntry(t.value); setResult(t.value) }
      if (h.status==='fulfilled') setHistory(h.value ?? [])
      if (hl.status==='fulfilled') setHolidays(hl.value ?? [])
    }).finally(() => setLoading(false))
}, [])
```

සිංහල: Page load කළ විට ස්වයංක්රීයව:

- * getLifestyleToday() -- අද submit කළ entry ඇත් නම් load.
- * getLifestyleHistory() -- ගිය days history load.
- * getHolidays() -- user holidays load.

2.4 -- handleSubmit()

```
const handleSubmit = async (e) => {
  e.preventDefault()
  setSubmitting(true)
  try {
    const r = await submitLifestyle({
      mood: form.mood, workload: form.workload, sleepHours: form.sleepHours,
      energyLevel: form.energyLevel, socialInteraction: form.socialInteraction,
      exerciseDone: form.exerciseDone, screenTimeHours: form.screenTimeHours,
      waterCups: form.waterCups,
    })
    setResult(r); setTodayEntry(r)
    setHistory(prev => [r, ...prev.filter(e => e.id !== r.id)])
    localStorage.setItem('lm_schedule_needs_refresh', Date.now().toString())
  } catch (err) { console.error(err) } finally { setSubmitting(false) }
}
```

සිංහල -- Submit කිරීමේදී:

- * Form submit prevent default (page reload නොකර).
- * setSubmitting(true) -- button "Analysing..." ලෙස show.
- * submitLifestyle() -- backend API ලෙස lifestyle data POST.
- * Backend ML prediction ලැබුණි -- r ලෙස result.
- * setResult(r) -- ML result card UI ලෙස show.
- * localStorage signal -- SchedulePage refresh trigger.

2.5 -- handleRefreshSchedule()

```
const handleRefreshSchedule = async () => {
  if (!result) return
  setRefreshing(true)
  try { await generateWeeklySchedule(result.stressLevel) }
  catch { /* ignore */ } finally { setRefreshing(false) }
}
```

සිංහල: "Refresh Schedule" button ක්ලික් කළ විට -- backend ශීර්තය latest stress level ලෙස weekly schedule regenerate. SchedulePage ලෙස update.

2.6 -- Holiday Calendar Logic

```
const handleAddHoliday = async (date, type) => {
  const optimistic = { date, type, note: '' }
  setHolidays(prev => [...prev.filter(x => x.date !== date), optimistic])
  try {
    const h = await addHoliday(date, type)
    setHolidays(prev => prev.map(x => x.date === date ? h : x))
  } catch { setHolidays(prev => prev.filter(x => x.date !== date)) }
  refreshScheduleAfterHolidayChange()
}
```

සිංහල:

- * Optimistic UI -- backend response ලොකු ගන්නා කලින් UI ලෙස ස්වයංක්රීයව update. User ක්ලික් -> ස්වයංක්රීය response ලෙස date colored.
- * Backend ශීර්තය fail -- UI rollback.
- * Schedule regenerate -- Holiday දිනය "Rest Day" ලෙස schedule.

2.7 -- ScaleSelector Sub-component

```
function ScaleSelector({ value, onChange, labels }) {
  return (
    <div className="flex gap-1.5 overflow-x-auto pb-0.5">
      {labels.map((label, i) => (
        <button key={i} type="button" onClick={() => onChange(i + 1)}
          className={`... ${value === i + 1 ? 'bg-violet-600/30 border border-violet-500/50' : ''}}>
          {label}
        </button>
      ))}
    </div>
  )
}
```

සිංහල: Workload, Energy, Social Interaction selectors සෑදීමේ reusable component. 1-5 scale buttons. Selected button violet glow ලෙස highlight.

2.8 -- ML Result Card (UI)

```
{result && (
  <div className={`mb-5 p-5 rounded-2xl border ${sc}`}>
    <div className="flex items-center gap-2">
      <Brain size={18}/> <span>ML Stress Analysis Result</span>
    </div>
    <p className="text-2xl font-extrabold">{result.stressLevel}</p>
    <p className="text-[12px]">{STRESS_MSG[result.stressLevel]}</p>
  </div>
)}
```

සිංහල: ML prediction ලැබෙයි නම් -- stress level, index, message card UI. Stress ශීර්ෂයේ ලෙස color badge ලෙස result.

2.9 -- Daily Check-in Form

Form fields:

Field	UI Element	Data
Mood	Emoji buttons (5)	mood: 1-5
Sleep Hours	± buttons + progress bar	sleepHours: 0-12
Workload	ScaleSelector (5 labels)	workload: 1-5
Energy Level	ScaleSelector (5 labels)	energyLevel: 1-5
Social Interaction	ScaleSelector (5 labels)	socialInteraction: 1-5
Exercise	Yes/No buttons	exerciseDone: 0 or 1
Screen Time	± 0.5 hr stepper	screenTimeHours: 0-14
Water Cups	± 0.5 cup stepper	waterCups: 0-16

3. WellnessPage.jsx -- රෝගා රෝගා පැහැදිලි කිරීම

3.1 -- Water Intake Tracker

```
const handleLogWater = async (amt) => {
  try { await logWater(amt); await reload(); setWaterFlash(true); ... }
  catch { /* ignore */ }
}
```

සිංහල: "+1 cup", "+2 cups", "+3 cups" buttons -- ක්ලික් කළ විට backend ශීර්ෂය water record save. reload() -- progress circle update.

3.2 -- Water Progress Circle

```
<circle cx="50" cy="50" r="42" fill="none" stroke="url(#wg)" strokeWidth="8"
strokeDasharray={ `${pct * 2.639} 264` } />
```

සිංහල: SVG circle -- ජල කෝෂ්ඨ 8 ලක්ෂ progress circle. $pct = (todayWater / 8) * 100$. 8 cups = 100% fill.

3.3 -- Workout Logger

```
const handleLogWorkout = async () => {
  if (!workoutForm.durationMinutes) return
  await logWorkout({ workoutType: workoutForm.workoutType,
    durationMinutes: parseInt(workoutForm.durationMinutes) })
  await reload()
}
```

සිංහල: Workout type (Running, Yoga, etc.) + duration (minutes) submit -- backend save.

3.4 -- Personalized Suggestions (ML-based)

```
const SUGGESTIONS_BY_LEVEL = {
  'Very Low': [ ... 6 suggestions ],
  'Low': [ ... 5 suggestions ],
  'Normal': [ ... 5 suggestions ],
  'High': [ ... 6 suggestions ],
  'Very High': [ ... 6 suggestions ],
}
```

සිංහල: Latest lifestyle data ML prediction ශ්‍රේණිගත stress level -> personalized suggestions show. High stress -> rest, breathe, water suggestions. Very Low stress -> intense workout, gratitude suggestions.

4. User Input Flow -- Frontend -> Backend -> ML Model

User (LifestylePage.jsx)

- ✓ form fill (mood, workload, sleep, energy, social, exercise, screen, water)
- ✓ Submit button click -> handleSubmit()
- ✓ submitLifestyle() -> POST /api/lifestyle (Spring Boot backend)

Spring Boot Backend (AuthService / LifestyleController)

- ✓ User authenticated (JWT token validate)
- ✓ Lifestyle data MongoDB save
- ✓ ML Service POST http://localhost:5001/predict (8 features)

ML Service (app.py FastAPI)

- ✓ Pydantic validate (ranges check)
- ✓ RandomForestClassifier.predict() -> stress_level (0-4)
- ✓ predict_proba() -> probabilities
- ✓ JSON response -> { stress_level, stress_label, probabilities }

Spring Boot Backend

- ✓ stress result save to Lifestyle record (MongoDB)
- ✓ WeeklyScheduleService.generateForCurrentWeek() trigger
- ✓ Weekly schedule build (stress level + user profile)
- ✓ Schedule MongoDB save
- ✓ Response -> Frontend { stressLevel, stressIndex, submittedAt }

Frontend (LifestylePage.jsx)

- ✓ setResult(r) -> ML result card show
- ✓ localStorage signal -> SchedulePage refresh
- ✓ User sees: stress level, message, schedule updated

ලේඛනය -- LifeMate ව්‍යාපෘතිය, Yasithzz, 2026